

MATEO HUANCHO MARIÑO

Backend Java Senior Developer | Full Stack Developer | Angular | Android | Microservicios | IAM & BPMN

Lima, Perú · Disponible en remoto

+51 918 076 179 · mhuancho@gmail.com

linkedin.com/in/mhuancho08 · github.com/mhuancho · huancho.dev

PERFIL PROFESIONAL

Backend Java Senior Developer con **8 años de experiencia** en diseño, desarrollo, modernización y estabilización de soluciones empresariales. Especializado en Java, Spring Boot, Quarkus, Micronaut, Keycloak, microservicios, APIs contract-first, arquitectura hexagonal, Clean Architecture, seguridad IAM, integración de APIs, mensajería y automatización de procesos con Camunda BPM.

Actualmente trabajo sobre **Java 21 con hilos virtuales, Spring Boot 4 y Quarkus 3.30**, en una plataforma multiempresa de facturación electrónica donde **creé de cero 12 de sus 24 repositorios**: la capa de identidad sobre Keycloak, las librerías corporativas de contratos y de resiliencia, cuatro API gateways y el procesamiento asíncrono de documentos.

Foco principal en backend y arquitectura de software, con experiencia complementaria en Angular y Micro-Frontends para colaborar en plataformas desacopladas, seguras y mantenibles. Perfil orientado a buenas prácticas, calidad, trazabilidad, contratos técnicos, reutilización de componentes y evolución sostenible de sistemas críticos.

ESPECIALIZACIÓN Y PROPUESTA DE VALOR

- **Arquitectura backend y solución por dominio:** Arquitectura Hexagonal (Ports & Adapters), Clean Architecture, separación de casos de uso, puertos/adaptadores, contratos, persistencia e infraestructura para construir soluciones mantenibles y evolutivas.
- **Microservicios empresariales y multi-tenant:** servicios desacoplados, reutilizables y seguros, orientados a dominio, con APIs REST contract-first (OpenAPI), GraphQL, resiliencia, mantenibilidad y escalabilidad operativa.
- **Seguridad avanzada e IAM:** Keycloak, OAuth2, OIDC, JWT, MFA/TOTP/OTP, SPI personalizados, RBAC, claims, roles, grupos, auditoría, autorización transversal y gestión centralizada de identidad.
- **Capa de experiencia (BFF) y API gateways:** un gateway por canal con rutas declarativas, filtros globales de trazabilidad y enrutamiento por empresa, resolución de tenant cacheada y edge sin estado.
- **Resiliencia como componente reutilizable:** política de circuit breaker, retry, bulkhead y time limiter empaquetada como librería, más un gateway dedicado que aísla los sistemas heredados.
- **Programación reactiva e integración frontend desacoplada:** Spring WebFlux, Project Reactor, R2DBC, RxJS, Angular, Micro-Frontends con Native Federation, guards, interceptores, refresh token y portalización dinámica.
- **Modernización de plataformas empresariales:** diseño de soluciones backend escalables y seguras, estandarización de componentes técnicos reutilizables, reducción de duplicidad y evolución sostenible de sistemas críticos.
- **Procesos BPMN, workflows y automatización empresarial:** Camunda 7/8, workers, correlación de mensajes, trazabilidad de estados, SLA, automatización con N8N e integración low-code/no-code para orquestación empresarial.
- **Procesos y eventos de integración:** Outbox Pattern, Kafka, RabbitMQ, Redis, eventos de dominio, sincronización entre APIs, BPM y servicios externos, con foco en consistencia transaccional y trazabilidad.
- **Procesamiento asíncrono con garantías:** idempotencia persistida, dead-letter queues con republicación, reencolado técnico programado y procesamiento por bloques para cargas masivas.
- **Calidad técnica y buenas prácticas:** JUnit 5, Mockito, JaCoCo, SonarQube, principios SOLID/DRY, manejo global de errores, respuestas estándar, documentación técnica, contratos y librerías corporativas reutilizables.

COMPETENCIAS TÉCNICAS

Lenguajes: Java 8/11/17/21 (hilos virtuales, records, pattern matching), TypeScript, JavaScript, SQL/PLSQL, .NET, Node.js

Backend: Spring Boot 3 y 4, Spring Cloud, Spring Cloud Gateway, Spring Cloud Stream, WebFlux, Quarkus 3, Micronaut, Java EE / Jakarta EE, JAX-RS, MicroProfile, JPA, Hibernate, MyBatis, Panache, Micronaut Data JDBC

Arquitectura: Arquitectura Hexagonal, Clean Architecture, DDD, microservicios, capas, MVC, event-driven, CQRS, BFF / capa de experiencia, librerías compartidas multi-framework, SOLID/DRY

APIs / Integración: REST, OpenAPI/Swagger, contract-first, GraphQL, Feign, WebClient, servicios externos, SOAP, BPMN, workflows

Seguridad: Keycloak 26, SPI (Authenticator, FormAction, EmailSender), Admin API, Themes FreeMarker, MFA/TOTP, OAuth2, OIDC, JWT, JWKS, RBAC, claims, roles, grupos, multi-tenancy, API keys, CORS, provisioning Just-In-Time

Resiliencia: Resilience4j (circuit breaker, retry, bulkhead, time limiter), fallbacks, backpressure, timeouts, graceful shutdown, tests de resiliencia

BPM / Automatización: Camunda 7, Camunda 8, BPMN, DMN, workers, mensajes, timers, eventos, N8N

Datos / Mensajería: PostgreSQL, Oracle, SQL Server, MySQL, MongoDB, Redis, SingleStore, Kafka, RabbitMQ, Spring Cloud Stream, R2DBC, Flyway, Outbox Pattern, dead-letter queues, idempotencia

Cloud / DevOps: Azure, AWS (S3, SES, CodeCommit), AKS, Docker, Kubernetes, Jenkins, GitLab CI/CD, Suite Atlassian

Observabilidad: Spring Boot Actuator, Micrometer, Prometheus, health checks liveness/readiness, correlación por cabecera, logging estructurado

Frontend: Angular 11-21, RxJS, Signals, Standalone Components, Nx monorepo, Native Federation, TailwindCSS, React, keycloak-js

Mobile: Android Studio, Android SDK, consumo de APIs REST, geolocalización, mantenimiento y evolución de aplicaciones Android

Testing / Calidad: JUnit 5, Mockito, RestAssured, JaCoCo, SonarQube, Karma, Jasmine, Vitest, Playwright

Complexless

Analista Programador Backend Java Senior | Mayo 2024 - Actualidad

Responsable del diseño y desarrollo de soluciones empresariales con foco en backend, seguridad, integración, arquitectura, estandarización técnica, microservicios, multi-tenancy, IAM, BPMN y plataformas desacopladas backend/frontend. Plataforma multiempresa de facturación electrónica: ~23 servicios backend y 4 aplicaciones Angular 21.

1. Solución IAM, autenticación multifactor y gestión centralizada de usuarios

Stack / prácticas: Java 21, Quarkus 3.30, Spring Boot 4, Keycloak 26, OAuth2/OIDC/JWT, PostgreSQL y SQL Server, Angular 21, arquitectura hexagonal, hilos virtuales

- Solución orientada a fortalecer autenticación, autorización, administración de usuarios y experiencia de acceso en una plataforma empresarial multi-cliente.
- Implementé servicios backend desacoplados para gestión de usuarios, perfiles, permisos y autorización, separando dominio, casos de uso, puertos, adaptadores y contratos.
- Extendí Keycloak mediante SPI personalizados para incorporar validaciones corporativas en el flujo de login, incluyendo validación obligatoria de RUC y reglas de acceso por cliente.
- Integré usuarios legacy con Just-In-Time Provisioning vía JDBC, soportando validación de contraseñas MD5/SHA-256 y comparación en tiempo constante para reducir riesgos de timing attack.
- Implementé MFA TOTP/OTP con enrolamiento QR, soporte multi-dispositivo, internacionalización y accesibilidad para flujos de autenticación reforzada.
- Estandaricé claims JWT hacia authorities y roles consumibles por microservicios, facilitando autorización consistente entre aplicaciones.
- **Añadí un SPI FormAction de preregistro de usuarios secundarios** con validación de empresa, control de duplicados y mensajes propios, y un **SPI EmailSenderProvider** que hace que todo el correo del proveedor de identidad salga por el servicio de correo de la plataforma en lugar de un SMTP aparte.
- **Desarrollé dos temas de login en FreeMarker** (Keycloak 26) con enrolamiento TOTP por QR y modo manual, selección entre varios dispositivos OTP, **9 idiomas** y **branding resuelto por dominio de acceso**, de modo que cada empresa ve su propia identidad visual.
- **Saqué la migración de la empresa completa del hilo de petición ejecutándola en hilos virtuales (Java 21)**, y automaticé la asignación de roles y grupos vía Admin API según los productos contratados.

2. Ecosistema empresarial modular multi-tenant con Quarkus

Stack / prácticas: Java 21, Quarkus 3.30/3.31, PostgreSQL y SQL Server, Panache, Flyway, Keycloak, OpenAPI, librerías corporativas

- Solución orientada a construir un ecosistema de servicios desacoplados para administración, onboarding, autorización empresarial y operación multi-cliente.
- Apliqué diseño por dominio y arquitectura por capas para aislar reglas de negocio, persistencia, seguridad, contratos REST y adaptadores técnicos.
- Implementé patrones reutilizables de paginación, metadata, respuestas tipadas, validaciones, manejo global de errores y seguridad transversal.
- Diseñé librerías corporativas para ApiResponse<T>, paginación, excepciones, validación JWT, conversión de claims a authorities y componentes base de workflows.
- Reduje duplicidad entre servicios y elevé consistencia técnica en autenticación, autorización, contratos, errores y estructura de respuesta.
- **Hice la librería corporativa deliberadamente agnóstica de framework**: el contexto multi-tenant se construye desde los claims sin depender de Quarkus ni de Spring, así que los servicios de ambos stacks comparten el mismo contrato.
- **Empaqueté la cadena de seguridad como autoconfiguración activable con una anotación** (@EnableKeycloakSecurity): validación JWT contra JWKS, verificación de issuer, conversión de roles de resource_access, rutas públicas y rol requerido configurables — eliminando la configuración de seguridad duplicada en más de 10 servicios.
- **Lideré la evolución de la API de seguridad a una v2** en repositorio nuevo, sustituyendo el acceso JDBC manual por entidades Panache, versionando el esquema con **Flyway**, separando los clientes de identidad y correo, e incorporando tareas programadas — sin interrumpir la v1 en producción.

3. Capa de seguridad e ingesta de documentos electrónicos

Stack / prácticas: Java 21, Quarkus 3.30, JAX-RS, hilos virtuales, API keys, OpenAPI contract-first, PostgreSQL y SQL Server

- Construí la puerta de entrada autenticada que recibe los documentos electrónicos y los reenvía a los sistemas de proceso, con autenticación por **API key por empresa** para integradores externos y por token OIDC para el portal.
- Implementé la **subida multipart en streaming** hacia fichero temporal con permisos restringidos, en lugar de bufferizar el archivo en memoria, eliminando el consumo de memoria proporcional al tamaño del paquete.
- Ejecuté los recursos de proxy sobre **hilos virtuales** (@RunOnVirtualThread), de modo que la espera de I/O contra los sistemas remotos no bloquea el pool de peticiones.
- Unifiqué el tratamiento de errores con mapeadores de excepción, de forma que todos los fallos salen con el mismo contrato de respuesta, y versioné el contrato **OpenAPI dentro del repositorio** como fuente de verdad para los integradores.

4. Capa de experiencia (BFF) y API gateways multi-tenant

Stack / prácticas: Spring Boot 4, Spring Cloud Gateway (WebFlux y WebMVC), WebClient reactivo, OAuth2 Resource Server, Micrometer/Prometheus

- Creé **cuatro gateways, uno por audiencia** (portal interno, canal público, integradores externos y resiliencia), en vez de un gateway único: cada canal tiene su CORS, sus timeouts, su superficie expuesta y su forma de autenticarse.
- Declaré las rutas por configuración en lugar de por código, de modo que añadir un destino no obliga a recompilar.
- Desarrollé filtros globales propios de **trazabilidad** (propagación de cabecera de correlación), **enrutamiento downstream por empresa** y cabecera de contexto para los servicios de dominio.
- Mantuve el **edge sin base de datos**: la resolución de la empresa se pide por HTTP al servicio de configuración, con **caché TTL de tamaño acotado y una sola consulta en vuelo por empresa**, para que al caducar una entrada en hora punta no se dispare una avalancha de consultas idénticas.
- Afiné pools de conexiones, timeouts y shutdown graceful, y restringí Actuator a health, info y prometheus con sondas de liveness/readiness.

5. Resiliencia transversal: librería de circuit breaker y gateway de aislamiento

Stack / prácticas: Java 21, Resilience4j, hilos virtuales, Spring Cloud Gateway, Micrometer, Prometheus

- Diseñé una **librería de resiliencia reutilizable** con una sola operación pública, que compone en orden **retry → time limiter → circuit breaker → bulkhead**: así los reintentos caben dentro del presupuesto de tiempo y el breaker ve el resultado final, no cada intento.
- Traté el **HTTP 429 como backpressure y no como fallo**, para no abrir el circuito justo cuando el sistema remoto está pidiendo bajar el ritmo.
- Modelé la política como un tipo que **valida su propia configuración al construirse**, de forma que una configuración incoherente falla al arrancar y no en producción; publiqué métricas por instancia en Micrometer.
- Construí además un **gateway dedicado con 7 circuit breakers nombrados** y dos perfiles de política (operaciones normales y de larga duración) que aísla los sistemas heredados, con **fallback HTTP** que responde Retry-After en lugar de dejar morir la petición.
- Cubrí el comportamiento con **tests de integración que validan el gateway con el upstream caído**.

6. Procesamiento asíncrono de comprobantes por lotes

Stack / prácticas: Spring Boot 4.1, Spring Cloud Stream, RabbitMQ, AWS S3, PostgreSQL, hilos virtuales, JDBC

- Saqué del camino síncrono la carga de paquetes con miles de comprobantes: el productor guarda el paquete en almacenamiento de objetos y publica **solo el evento** en la cola, respondiendo de inmediato al cliente.
- Implementé **idempotencia persistida en base de datos**, de modo que un reintento no duplica lo ya confirmado y sí reintenta lo que falló — la única garantía real de "exactly once" es un consumidor idempotente.
- Configuré **dead-letter queue con republicación**, tres intentos, concurrencia acotada y prefetch: 1 para que un consumidor lento no acapare mensajes, más una función que consume la DLQ y un **reencolado técnico programado**.
- Procesé **por bloques acotados (1.000 comprobantes)** republicando el evento con el resto del paquete, para que ningún mensaje individual agote el tiempo de confirmación.
- Expuse un endpoint interno de reencolado para operación y dejé trazabilidad de cada bloque en base de datos.

7. Servicios de dominio: facturación, planes, compras y documentos

Stack / prácticas: Spring Boot 4, JPA/Hibernate, R2DBC, WebFlux, MapStruct, Apache POI, JasperReports, AWS S3 y SES

- Desarrollé el servicio de **gestión de facturación y planes por cliente**, el mayor de la plataforma: **118 endpoints REST, 30 entidades**, generación masiva de comprobantes, dashboard de facturación y proceso por lotes.
- Construí el **registro de compras electrónico reactivo** con WebFlux y **R2DBC**, con routers funcionales y persistencia sobre **dos motores (PostgreSQL y SQL Server)**.
- Implementé el servicio de configuración y datos maestros que actúa como fuente de verdad para los gateways, y el servicio de correo corporativo sobre **AWS SES**, consumido también por el proveedor de identidad.
- Aporté a la generación documental (reportes, PDF y códigos QR) y a la exportación a hojas de cálculo con Apache POI.

8. Plataformas de micro-frontends con Nx y Native Federation

Stack / prácticas: Angular 21, Nx 22, Native Federation, Signals, Standalone Components, TailwindCSS 4, keycloak-js

- Participé en un monorepo con un *shell* anfitrión y **10 remotos federados por dominio** (documentos, trazabilidad, analítica, facturación, identidad, administración de empresas, reportes, configuración y compras) más 6 librerías compartidas.
- Definí un **contrato de navegación**: cada remoto expone sus ítems de menú y el *shell* los fusiona, deduplica y **filtra por los entitlements** del usuario calculados desde el token — nadie edita un menú central.
- Desarrollé la **librería compartida de contexto de autenticación** con estado reactivo (signals), que expone consultas de permiso en lugar de que cada remoto interprete los claims por su cuenta.
- Compartí dependencias como *singleton* con versión estricta para no duplicar el framework en el bundle, y compilé el CSS por remoto sobre una base común de utilidades.
- Desarrollé además dos aplicaciones Angular 21 independientes (administración y documentos) con cobertura de pruebas integrada en SonarQube.

9. Solución transaccional comercial, control de stock, auditoría y eventos de integración

Stack / prácticas: Java 21, Micronaut 4, Micronaut Security, Keycloak, PostgreSQL, Flyway, HikariCP, Kafka, Docker, GraalVM Native Image, Angular 21

- Solución backend para procesos comerciales con consistencia transaccional entre operación, stock, auditoría, trazabilidad y eventos de integración.
- Diseñé reglas de negocio para controlar estados, validar disponibilidad, evitar operaciones inválidas y mantener integridad entre entidades críticas.
- Implementé trazabilidad operativa mediante auditoría de usuario autenticado, timestamps, estados funcionales y registro de movimientos críticos.
- Apliqué Outbox Pattern para desacoplar la transacción principal del envío posterior de eventos, preservando consistencia entre base de datos y mensajería.
- Implementé publicación programada, reintentos, control de estados PENDING/PUBLISHED/ERROR y preparación para integración event-driven con Kafka.
- Configuré seguridad con JWT/JWKS, autorización por roles, rutas públicas/privadas, Micronaut Data JDBC, HikariCP, Flyway y preparación Docker/Native Image.

10. Sistema de solicitud de crédito con BPM y automatización de decisiones

Stack / prácticas: Java 21, Spring Boot 3, Camunda BPM, PostgreSQL, Angular 17+, arquitectura hexagonal

- Solución empresarial para digitalizar el ciclo de evaluación, aprobación y seguimiento de solicitudes de crédito.
- Desarrollé backend bajo arquitectura hexagonal, separando dominio, orquestación BPM, persistencia, integraciones y adaptadores técnicos.
- Implementé procesos BPMN con reglas de evaluación automática de riesgo, decisiones condicionales, SLA y escalamiento automático.
- Integré notificaciones HTML, eventos y actualización de estados para mejorar trazabilidad y visibilidad del ciclo operativo.
- Buenas prácticas aplicadas: desacoplamiento entre proceso y dominio, trazabilidad de estados, control de errores, pruebas y separación de responsabilidades.

11. Orquestación BPMN con Camunda 8 Self-Managed para gestión de pedidos

Stack / prácticas: Java 21, Spring Boot 4, Camunda 8, Zeebe, PostgreSQL, API REST

- Solución de orquestación para gestionar pedidos mediante estados de negocio, eventos externos y automatización de tareas.
- Desarrollé API REST para iniciar procesos, gestionar eventos de negocio y persistir trazabilidad del pedido en PostgreSQL.
- Configuré correlación de mensajes para avanzar flujos desde sistemas externos y workers para automatización, validación y cierre del proceso.
- Apliqué desacoplamiento entre lógica de negocio, persistencia y motor BPMN, validando el flujo desde API, motor y monitoreo operativo.

12. Optimización operativa portuaria - Terminal Portuarios Peruanos (TPP)

Stack / prácticas: Spring Boot 3, Angular 19, AWS SDK, MyBatis, SQL Server, SonarQube, JUnit 5, JaCoCo

- Solución orientada a optimizar procesos operativos y de control portuario mediante validaciones tempranas, automatización y trazabilidad.
- Implementé validaciones de cut-off para evitar ingresos indebidos al flujo operativo y mejorar control previo de operaciones críticas.
- Desarrollé pre-citas automatizadas, QR dinámicos y alertas operativas periódicas para mejorar seguimiento y eficiencia operacional.
- Participé en mejoras que redujeron aproximadamente 40% de errores operativos, reforzando estabilidad y calidad del proceso.
- Buenas prácticas aplicadas: pruebas unitarias, análisis estático, cobertura, documentación técnica y mantenimiento evolutivo controlado.

13. VUCE 2.0 - Ventanilla Única de Comercio Exterior y procesos portuarios

Stack / prácticas: Spring Boot 3, Java 17, Quarkus, MapStruct, JasperReports, Gradle, Lombok, Angular 17, Redis, Kafka, Feign, Camunda BPM

- Solución de transformación digital para procesos de comercio exterior, operaciones portuarias, trazabilidad de trámites y automatización documental.
- Diseñé microservicios contract-first con OpenAPI y arquitectura hexagonal, incorporando capa de experiencia para mejorar organización, escalabilidad y mantenibilidad.
- Implementé procesos para embarcaciones de entrada/salida, seguimiento de trámites, carga masiva y documentación técnica de integración.
- Integré firma digital para Autorización de Zarpe / Port Clearance, reportes JasperReports, pasarela de pago, Redis, Kafka, Feign y Camunda BPM.
- Contribuí con diagramas de secuencia, contratos, pruebas funcionales/técnicas y librerías frontend reutilizables para estandarización Angular.

Sagone - Techera

Java Developer | Octubre 2018 - Abril 2024

Participación en desarrollo, evolución e integración de soluciones empresariales para sectores regulados y organizaciones con operación continua. Foco en backend Java, procesos BPMN, integración documental, interoperabilidad, rendimiento, estabilidad y mantenimiento evolutivo, incluyendo desarrollo mobile Android.

14. Sistema de Transmisión Eléctrica (STE) - OSINERGMIN

Stack / prácticas: Java, Spring Boot, Quarkus, Camunda 7, Redis, bases de datos relacionales

- Solución regulatoria para procesos técnicos y económicos del Sistema de Transmisión Eléctrica, con soporte a planificación, valorización, COyM, SST y SCT.
- Lideré implementación de procesos BPMN con Camunda 7 para automatizar flujos regulatorios y operativos del negocio eléctrico.
- Integré procesos BPMN, servicios backend y persistencia para asegurar trazabilidad, ejecución controlada, transición de estados y seguimiento del proceso.
- Desarrollé reglas de negocio, cálculos, validaciones, consultas y procesamiento de información técnica, económica y regulatoria.
- Optimicé procesos de carga y consulta utilizando Redis como caché para información temporal asociada a archivos y operaciones intensivas.
- Aseguré continuidad operativa mediante mantenimiento evolutivo/correctivo de sistemas críticos alineados a cambios normativos.

15. Proyecto FACILITO de OSINERGMIN - Aplicación móvil Android

Stack / prácticas: Android Studio, Android SDK, Java para Android, consumo de APIs REST, geolocalización, manejo de errores, mantenimiento evolutivo y UX móvil

- Participé en el desarrollo y mantenimiento evolutivo de FACILITO, solución móvil gratuita de OSINERGMIN orientada a que los usuarios ubiquen y comparen precios de combustibles y balones de gas en establecimientos formales.
- Implementé y ajusté funcionalidades móviles para consulta de información, navegación, filtros, listados, ubicación de establecimientos y visualización de precios actualizados desde servicios backend.
- Integré consumo de APIs REST y manejo de respuestas para presentar información confiable al usuario final, incorporando validaciones, control de errores y soporte a escenarios de conectividad móvil.
- Apliqué buenas prácticas de desarrollo Android: separación de responsabilidades, mantenibilidad del código, compatibilidad entre dispositivos, optimización de consumo de datos y cuidado de la experiencia de usuario.
- Contribuí a la estabilización, soporte y mejora continua de una aplicación ciudadana de alto uso, priorizando rendimiento, trazabilidad de incidencias y continuidad operativa.

16. Gestión documental, interoperabilidad y evolución de plataformas empresariales

Stack / prácticas: Java EE, Spring Boot, Quarkus, Angular, React, Alfresco, APIs REST/SOAP, Kafka, AWS, Docker, Kubernetes, Jenkins, GitLab CI/CD, Atlassian

- **Gestión documental integrada con Alfresco:** desarrollé integraciones para carga, descarga, búsqueda, clasificación documental y administración de metadatos, asegurando consistencia entre el sistema transaccional y el repositorio documental mediante reglas de acceso, trazabilidad y separación entre integración documental y lógica de negocio.
- **Interoperabilidad con RENIEC y servicios externos:** implementé conectores, validaciones, adaptación de respuestas y manejo de errores entre plataformas heterogéneas, separando contratos externos, mapeo de datos y reglas internas para reducir acoplamiento y facilitar el mantenimiento.
- **Plataformas de CRM, comercio electrónico, educación y control operativo:** desarrollé gestión de clientes, seguimiento comercial, catálogos, flujo de compra, registros académicos, stock y trazabilidad operativa, aportando mejoras de rendimiento, corrección de incidencias e integración entre capas.
- **Mantenimiento evolutivo y continuidad operativa de más de 50 sistemas en producción** con distintos niveles de criticidad: corrección de incidencias, ajustes funcionales y adaptación a nuevos requerimientos sin comprometer estabilidad, colaborando en la modernización progresiva con microservicios, arquitectura hexagonal, automatización de integración y despliegue continuo.

Solución

Practicante Programador Junior | Octubre 2015 - Octubre 2017

- Inicio profesional en desarrollo de aplicaciones empresariales con JSP, Servlets, patrón MVC, DAOs y consultas SQL.
- Apoyo en mantenimiento de aplicaciones, soporte técnico, QA funcional y pruebas técnicas.

BUENAS PRÁCTICAS TRANSVERSALES

- **Arquitectura:** separación de responsabilidades, diseño por dominio, puertos/adaptadores, contratos OpenAPI, capas limpias y límites claros entre servicios.
- **Seguridad:** OAuth2/OIDC/JWT, Keycloak, MFA, SPI, RBAC, validación de tokens, normalización de claims, auditoría de usuario y control por roles/grupos.
- **Resiliencia:** circuit breaker, retry, bulkhead, time limiter, fallbacks, timeouts, backpressure y aislamiento de dependencias heredadas.
- **Integración:** Outbox Pattern, reintentos, publicación asíncrona, idempotencia, dead-letter queues, Kafka, RabbitMQ, Redis, Feign, BPMN y automatización con N8N.
- **Calidad:** JUnit 5, Mockito, JaCoCo, SonarQube, manejo global de errores, respuestas estándar, validaciones centralizadas y reducción de duplicidad.
- **Operación:** trazabilidad, logging, métricas con Micrometer/Prometheus, health checks, documentación, diagramas de secuencia, versionamiento de base de datos y Docker/Kubernetes.

EDUCACIÓN

- **Ingeniería de Sistemas e Informática** — UTP · 2024 – Actualidad (convalidando cursos, ciclo III)
- **Titulado en Telemática** — IDAT · 2015 – 2017

IDIOMAS

- **Español:** nativo
- **Inglés:** intermedio — lectura y escritura técnica